



Интро

- ТЕХНИКА

Проверка связи

Если у вас нет звука

убедитесь, что на устройстве и колонках включён звук
обновите страницу или переподключитесь к вебинару
откройте вебинар в другом браузере
перезагрузите устройство и войдите заново

Пожалуйста, отметьтесь в чате



если меня видно и слышно



если есть проблемы со звуком или видео

- 01 / Десктоп

С компьютера

Используйте **Google Chrome** или **Microsoft Edge**.

При проблемах - обновите страницу **F5**

- 02 / Мобильный

С телефона или планшета

Перейдите с мобильного интернета на **Wi-Fi**.

При проблемах - перезапустите приложение Webinar.

- 03 / Общий совет

Если ничего не помогло

Отключитесь и подключитесь заново.

Пишите в чат - организаторы помогут.

- ЗНАКОМСТВО

Перед началом

- КАКИЕ НОВОСТИ?

Что интересного произошло за неделю?

Напишите в чат - событие, новость или вывод из работы, учёбы или личного опыта.



- ПРЕПОДАВАТЕЛЬ КУРСА

Павлин Николай

Преподаватель кафедры бизнес-информатики МТУСИ
с 2021 года

СТО Nova · стартап в сфере мониторинга здоровья · резидент Сколково

Разрабатываю **LLM-приложения** и RAG-платформу **RAGMaster** (грант ФСИ)

Автор RAG-системы для охраны труда - **chat.kiout.ru**, 1000+ пользователей

Соавтор LLM, попавшей в **топ-20** русскоязычных моделей по MERA (02.2025)

7+ лет в коммерческой разработке: Python, FastAPI, LLM-стек, DevOps

Многократный победитель федеральных хакатонов



Telegram



YouTube



LinkedIn

- ДОГОВОРЁННОСТИ

Правила участия

- 1** Продолжительность вебинара - **80 минут**
- 2** Задавайте **вопросы в чате** - ответу в процессе или в конце
- 3** **Запись** будет доступна на платформе на следующий день
- 4** Обсуждение можно **продолжить в чате** после вебинара
- 5** **Воздерживаемся от обсуждения политических тем.** Здесь мы про разработку, ИИ и продукты - давайте сохраним фокус.

Начинаем!





Асинхронная обработка запросов к ИИ

Модуль 3 Блок 3

Павлин Н.К.

План занятия

Блок	Что разбираем
Вход	Outcomes, опрос «сколько у вас async-кода»
Теория	Concurrency vs parallelism, I/O-bound, event loop
Теория	<code>async</code> / <code>await</code> , корутины, Task, Future
Теория	<code>gather</code> , <code>Semaphore</code> , <code>TaskGroup</code> , <code>asyncio.timeout</code>
Демо-код #1	Sync → async: M2-клиент через <code>AsyncOpenAI</code> + <code>gather</code> + <code>Semaphore</code>
Демо-код #2	Async streaming от LLM + SSE-эндпоинт на FastAPI
Теория/демо	Пул провайдеров, таймауты на 3 уровнях, background tasks
Демо-код #3	Бенчмарк sync vs async (1/5/10/50 параллельных запросов)
Q&A + ДЗ	Типичные ошибки, обсуждение, ДЗ

Что вы уже знаете

Из M2B3: синхронный LLM-клиент с `retry`, `fallback`, таймаутами. Сегодня переводим его на `async`.

Из M2B4: кеширование в Redis с ключом по промπτу. Async-клиент будет ходить в тот же кеш.

Из M3B1: function calling — полный цикл `tool_call`. Async усиливает: параллельные `tool_calls` через `gather`.

Из M3B2: паттерны **Streaming, Fan-out/Fan-in, Circuit Breaker, Bulkhead**. Сегодня — их реализация в коде через `asyncio`.

Чего ещё не было:

- `asyncio.TaskGroup` и `asyncio.timeout` (Python 3.11+) — современный путь
- SSE от LLM до браузера через FastAPI
- Реальные бенчмарки `sync` vs `async` на ваших промптах

Опрос: async в ваших проектах

Проголосуйте в чате цифрой:

- 1** — async-кода нет совсем, везде `def`
- 2** — есть кое-где, но не понимаю, как работает event loop
- 3** — всё async, использую `TaskGroup` и `Semaphore`

Словарь занятия

Термин	English	Что это
Конкурентность	Concurrency	Несколько задач выполняются *с переключением*, не обязательно одновременно
Параллелизм	Parallelism	Несколько задач выполняются *физически одновременно* на разных ядрах
I/O-bound	I/O-bound	Задача ограничена ожиданием сети/диска, а не процессором
CPU-bound	CPU-bound	Задача ограничена вычислениями, <code>async</code> не поможет
Event Loop	Event Loop	Цикл-диспетчер, переключается между готовыми задачами
Корутина	Coroutine	Функция с <code>async def</code> , «засыпает» на <code>await</code>
Task	Task	Обёртка над корутиной, запущенная в event loop
Future	Future	Объект-обещание результата, низкий уровень
<code>gather</code>	<code>asyncio.gather</code>	Запустить N корутин параллельно и дождаться всех
TaskGroup	<code>asyncio.TaskGroup</code>	Structured concurrency (3.11+): все задачи в одном блоке
Semaphore	<code>asyncio.Semaphore</code>	Счётчик, ограничивает число одновременных задач
<code>asyncio.timeout</code>	<code>asyncio.timeout</code>	Контекст-менеджер таймаута (3.11+), замена <code>wait_for</code>
AsyncIterator	<code>AsyncIterator</code>	Итератор, <code>async for</code> по нему
SSE	Server-Sent Events	Поток событий сервер → клиент по обычному HTTP

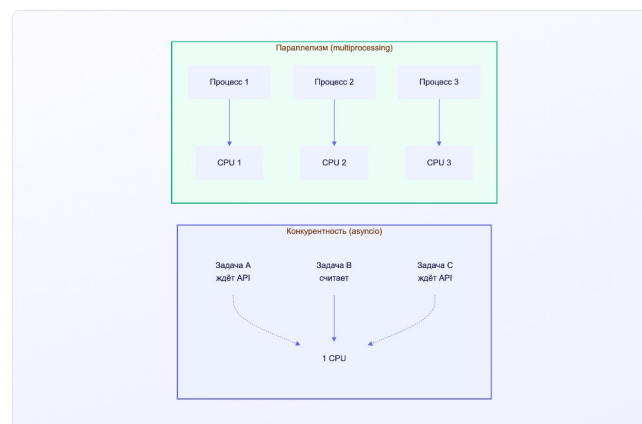
Concurrency vs parallelism

Конкурентность — несколько задач «в работе» одновременно, но не обязательно *выполняются* одновременно.

Параллелизм — несколько задач *физически* выполняются в один момент на разных ядрах/машинах.

Аналогия. Один повар, который варит суп (ждёт плиту), в это время режет салат, потом мешает суп — это конкурентность. Три повара на трёх станциях — это параллелизм.

Для LLM: вы на 99% ждёте сеть. Один повар (один поток) идеально справляется с сотней «варящихся супов».



Почему asyncio идеален для LLM

Один вызов LLM на GPT-4.1-mini выглядит так:

Вывод: **99.7% времени CPU простаивает**. Один поток может обслуживать сотни таких «простаивающих» запросов.

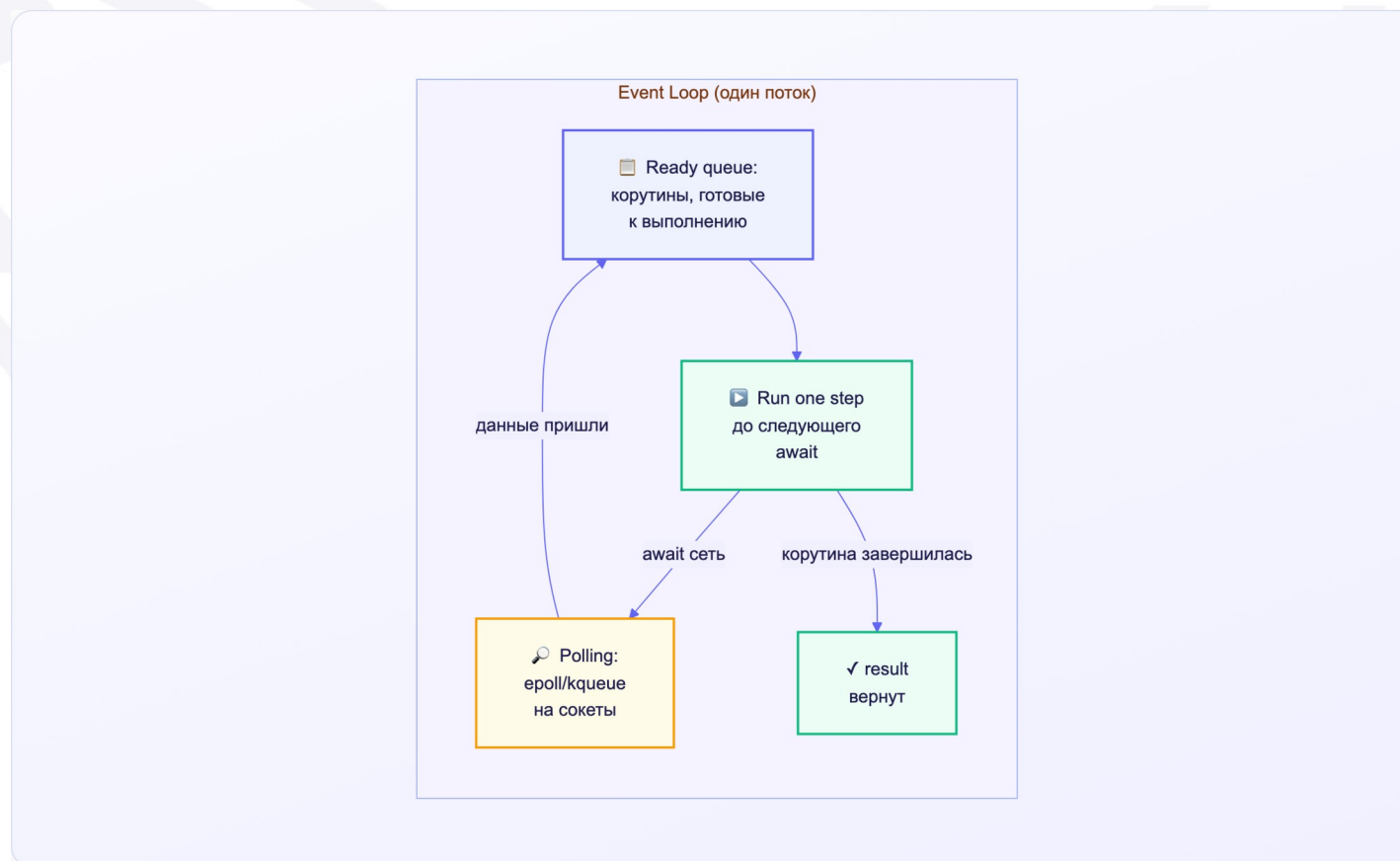
Что из этого следует:

- Sync с ThreadPool: 100 потоков × 8 МБ памяти = **800 МБ** просто «для ожидания»
- Async: 100 корутин × 2–4 КБ = **~300 КБ**
- Масштабирование: один FastAPI-воркер → 500+ одновременных LLM-запросов без страданий

Фаза	Время	CPU активен?
DNS + TCP + TLS	~50–100 мс	нет
Отправка запроса	~10 мс	нет
Ожидание первого токена (TTFT)	200–600 мс	нет
Стриминг токенов	3–10 секунд	нет
Парсинг JSON	~5 мс	да
Итого	~5 секунд	~15 мс CPU

Event Loop: упрощённая модель

Event Loop — бесконечный цикл, который обслуживает **очередь готовых задач** (ready queue) и **пул ожидающих** (waiting on I/O).



Event Loop: упрощённая модель

Три шага цикла:

1. Берём корутину из ready queue, выполняем до следующего `await`.
2. На `await sock.recv()` — регистрируем сокет в epoll, корутина «спит».
3. Когда epoll говорит «данные пришли» — возвращаем корутину в ready queue.

Всё в одном потоке. Всегда.

async/await: синтаксис и семантика

sync:

```
example.python

1  import time
2  from openai import OpenAI
3
4  client = OpenAI()
5
6  def ask(q: str) -> str:
7      r = client.chat.completions.create(
8          model="gpt-5-mini",
9          messages=[{"role": "user", "content": q}],
10     )
11     return r.choices[0].message.content
12
13 start = time.time()
14 answers = [ask(q) for q in ["1+1?", "2+2?", "3+3?"]]
15 print(f"Sync: {time.time() - start:.1f} сек") # ~15 сек (5+5+5)
```


async/await: синтаксис и семантика

Три запроса — три последовательных ожидания.

async/await: асинх-версия

```
example.python

1  import asyncio, time
2  from openai import AsyncOpenAI
3
4  client = AsyncOpenAI()
5
6  async def ask(q: str) -> str:
7      r = await client.chat.completions.create(
8          model="gpt-5-mini",
9          messages=[{"role": "user", "content": q}],
10         )
11     return r.choices[0].message.content
12
13  async def main():
14      start = time.time()
15      answers = await asyncio.gather(
16          ask("1+1?"), ask("2+2?"), ask("3+3?")
17      )
18      print(f"Async: {time.time() - start:.1f} сек") # ~5 сек
19
20  asyncio.run(main())
```

async/await: async-версия

Три изменения:

1. `OpenAI` → `AsyncOpenAI` (другой класс из того же SDK)
2. `def` → `async def`, перед вызовом SDK — `await`
3. Список → `asyncio.gather(*coros)` запускает всё параллельно

Корутина, Task, Future

Три связанных, но разных понятия.

```
example.python

1  import asyncio
2
3  async def f():
4      return 42
5
6  # Корутина — результат вызова async-функции, "рецепт"
7  coro = f()
8  print(type(coro))          # <class 'coroutine'>
9
10 # Task — корутина, запущенная в event loop
11 task = asyncio.create_task(f()) # стартует сразу
12 result = await task           # ждём, .cancel() для отмены
13
14 # Future — низкоуровневое "обещание", обычно создаёт библиотека
15 fut = asyncio.Future()
16 # fut.set_result(42) — низкий уровень, в прикладном коде почти не нужен
```

Корутина, Task, Future

Правило большого пальца:

- `await coroutine` — дождаться результата, ничего не запускать параллельно
- `asyncio.create_task(coroutine)` — запустить параллельно в фоне
- `asyncio.gather(*coros)` — запустить все параллельно и дождаться

asyncio.gather: параллельные запросы

Задача: обработать 10 отзывов LLM-анализом, параллельно.

```
example.python

1  import asyncio
2  from openai import AsyncOpenAI
3
4  client = AsyncOpenAI()
5
6  async def analyze(text: str) -> dict:
7      r = await client.chat.completions.create(
8          model="gpt-5-mini",
9          messages=[{"role": "user", "content": f"Оцени тон: {text}" }],
10         max_tokens=50,
11     )
12     return {"text": text[:40], "tone": r.choices[0].message.content}
13
14  async def batch(texts: list[str]) -> list[dict | Exception]:
15      coros = [analyze(t) for t in texts]
16      return await asyncio.gather(*coros, return_exceptions=True)
17
18  # Использование:
19  results = await batch(reviews)
20  ok = [r for r in results if not isinstance(r, Exception)]
21  err = [r for r in results if isinstance(r, Exception)]
22  print(f"ok={len(ok)}, errors={len(err)}")
```

asyncio.gather: параллельные запросы

Ключевое — `return_exceptions=True`:

- По умолчанию: первая ошибка выбрасывается, остальные задачи **не отменяются**, но результат теряется.
- С флагом: исключение возвращается как элемент списка, остальные задачи доходят до конца.

asyncio.Semaphore: ограничение конкурентности

Проблема. OpenAI Tier 1 на начало 2026: ~500 RPM $\approx$ 8 RPS. 100 одновременных запросов через `gather` → мгновенный 429.

```
example.python
1  import asyncio
2  from openai import AsyncOpenAI
3
4  client = AsyncOpenAI()
5  SEM = asyncio.Semaphore(8) # максимум 8 одновременных запросов
6
7  async def analyze(text: str) -> str:
8      async with SEM: # войти можно, только если есть свободный слот
9          r = await client.chat.completions.create(
10              model="gpt-5-mini",
11              messages=[{"role": "user", "content": f"Анализ: {text}"},],
12              max_tokens=50,
13          )
14          return r.choices[0].message.content
15
16  async def batch(texts: list[str]) -> list[str]:
17      return await asyncio.gather(
18          *[analyze(t) for t in texts],
19          return_exceptions=True,
20      )
```


asyncio.Semaphore: ограничение конкурентности

Как работает. `Semaphore(8)` = счётчик 8. `async with SEM` занимает слот (счётчик -1) или ждёт, если 0. На выходе — освобождает (+1). Result: **никогда** > 8 активных `await client.chat.completions.create`.

Как выбрать число. Правило: $\text{floor}(\text{RPM} / 60 \times 0.8)$.

asyncio.TaskGroup: structured concurrency (3.11+)

Проблема `gather`. Нет чёткого жизненного цикла задач: они могут «утекать» в фон, отмена работает криво при ошибках.

TaskGroup (добавлен в Python 3.11) — абстракция для объединения задач: все стартуют в блоке `async with`, все завершаются к выходу. Ошибка в одной — отменяются остальные.

```
example.python
1  import asyncio
2  from openai import AsyncOpenAI
3
4  client = AsyncOpenAI()
5
6  async def translate(text: str, lang: str) -> str:
7      r = await client.chat.completions.create(
8          model="gpt-5-mini",
9          messages=[{"role": "user", "content": f"→ {lang}: {text}" }],
10         max_tokens=500,
11     )
12     return r.choices[0].message.content
13
14  async def translate_all(text: str) -> dict[str, str]:
15      async with asyncio.TaskGroup() as tg:
16          en = tg.create_task(translate(text, "English"))
17          de = tg.create_task(translate(text, "German"))
18          fr = tg.create_task(translate(text, "French"))
19      # сюда попадаем только если ВСЕ три завершились
20      return {"en": en.result(), "de": de.result(), "fr": fr.result()}
```

asyncio.TaskGroup: structured concurrency (3.11+)

Если один перевод упал — TaskGroup автоматически отменяет два остальных и выбрасывает `ExceptionGroup` на выходе.

ExceptionGroup и except*

При ошибке в задаче TaskGroup выбрасывает **ExceptionGroup** — коллекцию всех исключений, которые произошли.

```
example.python

1  try:
2      async with asyncio.TaskGroup() as tg:
3          tg.create_task(call_openai())
4          tg.create_task(call_anthropic())
5          tg.create_task(call_local_llm())
6  except* TimeoutError as eg:
7      # обработка таймаутов из любой ветки
8      for e in eg.exceptions:
9          log.warning("timeout in provider", exc_info=e)
10 except* APIError as eg:
11     for e in eg.exceptions:
12         log.error("api error", status=e.status_code)
```

ExceptionGroup и except*

- `except*` — новый синтаксис Python 3.11+, ловит *подмножество* исключений из `ExceptionGroup`.
- Группа может содержать разнородные ошибки: `TimeoutError`, `APIError`, `ValueError` в одном запуске.

asyncio.timeout: современные таймауты (3.11+)

Старый способ — `asyncio.wait_for` оборачивает **одну корутину**.

Новый — `asyncio.timeout` как контекст-менеджер, оборачивает **любой блок** кода с несколькими `await`.

```
example.python

1  import asyncio
2  from openai import AsyncOpenAI
3
4  client = AsyncOpenAI()
5
6  # Старый стиль (до 3.11):
7  # result = await asyncio.wait_for(client.chat.completions.create(...), 10.0)
8
9  # Новый стиль (3.11+):
10 async def ask_with_timeout(q: str) -> str:
11     try:
12         async with asyncio.timeout(10.0):
13             r = await client.chat.completions.create(
14                 model="gpt-5-mini",
15                 messages=[{"role": "user", "content": q}],
16             )
17             # внутри блока может быть несколько await подряд
18             return r.choices[0].message.content
19     except TimeoutError: # обычный TimeoutError, не asyncio.TimeoutError
20         return "Извините, не успел. Попробуйте ещё раз."
```

asyncio.timeout: современные таймауты (3.11+)

Преимущества `asyncio.timeout`:

- Не создаёт лишний Task (быстрее `wait_for`).
- Охватывает несколько `await` подряд одним лимитом.
- Безопасно вкладывается: внешний лимит всегда «побеждает» внутренний.
- Единый `TimeoutError` — не `asyncio.TimeoutError` (в 3.11+ это алиасы).

От примитивов к SDK

Дальше — как всё это собирается в реальном LLM-клиенте.

Что было: примитивы `asyncio` — `gather`, `Semaphore`, `TaskGroup`, `timeout`, `ExceptionGroup`.

Что добавляем: `AsyncOpenAI` / `AsyncAnthropic`, обёртка вокруг них с лимитом конкурентности, переход с синхронного M2-клиента на `async`.

AsyncOpenAI: базовое использование

`openai>=2.0` включает асинхронный клиент прямо в пакет: `AsyncOpenAI` — зеркало обычного `OpenAI`, но все методы — `await`.

```
example.python

1 import asyncio
2 from openai import AsyncOpenAI
3
4 # max_retries и timeout — параметры клиента, не per-request
5 client = AsyncOpenAI(timeout=30.0, max_retries=2)
6
7 async def ask(q: str, model: str = "gpt-5-mini") -> str:
8     resp = await client.chat.completions.create(
9         model=model,
10        messages=[{"role": "user", "content": q}],
11    )
12    return resp.choices[0].message.content
13
14 answer = asyncio.run(ask("Объясни, что такое event loop"))
```

AsyncOpenAI: базовое использование

Что даёт AsyncOpenAI из коробки:

- Transparent retry на 429/5xx с exponential backoff
- Таймауты на уровне httpx
- Structured Outputs (`response_format`), Function Calling, Streaming — всё то же, но с `await`
- Можно одновременно использовать `OpenAI` (sync) и `AsyncOpenAI` в одном проекте

Sync → async клиент: разбор перехода

Превращение синхронного клиента из M2Б3 (`llm_client.py`) в async — четыре механических шага плюс одно архитектурное решение.

Шаги:

1. `from openai import OpenAI` → `from openai import AsyncOpenAI`
2. В методах класса: `def complete(...)` → `async def complete(...)`, добавить `await` перед вызовом `.chat.completions.create`
3. Добавить метод `async def batch_complete(self, prompts: list[str], max_concurrency: int = 8)` через `gather + Semaphore`
4. Обновить интерфейс кеша (M2Б4) — если кеш в Redis, использовать `redis.asyncio` вместо `redis`
5. Точка входа в скрипте — `asyncio.run(main())`

Sync → async клиент: разбор перехода

example.python

```
1 class AsyncLLMClient:
2     def __init__(self, model: str = "gpt-5-mini", max_concurrency: int = 8):
3         self._client = AsyncOpenAI()
4         self._sem = asyncio.Semaphore(max_concurrency)
5         self._model = model
6
7     async def complete(self, prompt: str) -> str:
8         async with self._sem:
9             r = await self._client.chat.completions.create(
10                 model=self._model,
11                 messages=[{"role": "user", "content": prompt}],
12             )
13             return r.choices[0].message.content
14
15     async def batch(self, prompts: list[str]) -> list[str | Exception]:
16         return await asyncio.gather(
17             *[self.complete(p) for p in prompts],
18             return_exceptions=True,
19         )
```

От batch к streaming

Теперь меняем модель «отдать всё разом» на «отдавать по мере появления».

Что было: batch-вызовы — пользователь ждёт полный ответ.

Что добавляем: async-streaming от LLM, SSE как транспорт от FastAPI до клиента, async-генераторы.

Async Streaming от LLM

`stream=True` на `chat.completions.create` — и метод возвращает **async iterator** по дельтам.

```
example.python

1 from openai import AsyncOpenAI
2
3 client = AsyncOpenAI()
4
5 async def stream_text(q: str):
6     stream = await client.chat.completions.create(
7         model="gpt-5-mini",
8         messages=[{"role": "user", "content": q}],
9         stream=True,
10        stream_options={"include_usage": True}, # usage в последнем chunk
11    )
12    async for chunk in stream:
13        if chunk.choices and chunk.choices[0].delta.content:
14            yield chunk.choices[0].delta.content
15        if chunk.usage: # финальный chunk
16            print(f"\n[tokens={chunk.usage.total_tokens}]")
17
18 async def main():
19     async for token in stream_text("Напиши хайку про async"):
20         print(token, end="", flush=True)
```

Async Streaming от LLM

- `async for chunk in stream` — итерация без блокировки event loop.
- `yield` в async-функции делает её **async-генератором**.
- `stream_options={"include_usage": True}` — OpenAI пришлёт **usage** в последнем пустом chunk'е (без этого флага при стриминге usage недоступен).

FastAPI + SSE-эндпоинт

Минимальный FastAPI-эндпоинт `POST /chat/stream`, прокидывающий LLM-стрим клиенту через SSE.

```
example.python

1 from fastapi import FastAPI
2 from pydantic import BaseModel
3 from fastapi.responses import StreamingResponse
4 from openai import AsyncOpenAI
5 from typing import AsyncIterable
6
7 app = FastAPI()
8 client = AsyncOpenAI()
9
10 class ChatRequest(BaseModel):
11     question: str
12     model: str = "gpt-5-mini"
```

```
example.python

1 async def sse_from_llm(req: ChatRequest) -> AsyncIterable[str]:
2     stream = await client.chat.completions.create(
3         model=req.model,
4         messages=[{"role": "user", "content": req.question}],
5         stream=True,
6     )
7     async for chunk in stream:
8         delta = (
9             chunk.choices[0].delta.content
10             if chunk.choices
11             else None
12         )
13         if delta:
14             yield f"event: token\ndata: {delta}\n\n"
15         yield "event: done\ndata: [END]\n\n"
16
17
18 @app.post("/chat/stream", response_class=StreamingResponse)
19 async def chat_stream(req: ChatRequest) -> AsyncIterable[str]:
20     async for chunk in sse_from_llm(req):
21         yield chunk
```

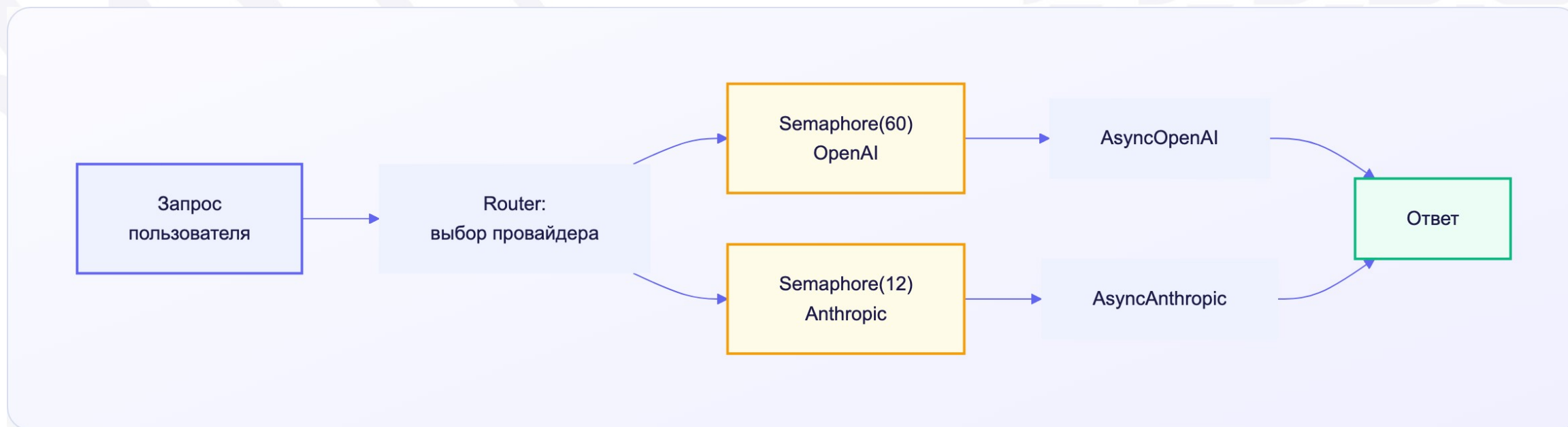

FastAPI + SSE-эндпоинт

Запуск: `uvicorn app:app --reload`. Проверка `curl -N -X POST ...` показывает «печатающийся» ответ в терминале.

Пул провайдеров с раздельными лимитами

Реальный кейс. У вас два провайдера: OpenAI (Tier 2 — 5000 RPM) и Anthropic (Tier 1 — 1000 RPM). Нельзя использовать один общий Semaphore — разные лимиты.

Идея. Каждый провайдер — свой клиент, свой Semaphore. Router выбирает по правилу (cost-based, latency-based, round-robin, racing).



Код: пул провайдеров

example.python

```
1 import asyncio
2 from dataclasses import dataclass
3 from openai import AsyncOpenAI
4 from anthropic import AsyncAnthropic
5
6
7 @dataclass
8 class Provider:
9     name: str
10    client: object
11    sem: asyncio.Semaphore
12    call: callable # async func prompt -> str
13
14
15 async def call_openai(client, prompt):
16     r = await client.chat.completions.create(
17         model="gpt-5-mini",
18         messages=[{"role": "user", "content": prompt}],
19     )
20     return r.choices[0].message.content
```

example.python

```
1 async def call_claude(client, prompt):
2     m = await client.messages.create(
3         model="claude-sonnet-4-6", max_tokens=1024,
4         messages=[{"role": "user", "content": prompt}],
5     )
6     return m.content[0].text
7
8
9 PROVIDERS = [
10     Provider("openai", AsyncOpenAI(), asyncio.Semaphore(60), call_openai),
11     Provider("claude", AsyncAnthropic(), asyncio.Semaphore(12), call_claude),
12 ]
13
14
15 async def ask(prompt: str, provider_name: str = "openai") -> str:
16     p = next(x for x in PROVIDERS if x.name == provider_name)
17     async with p.sem:
18         return await p.call(p.client, prompt)
```

Код: пул провайдеров

Использование: `await ask("Hello", "openai") / await ask("Hello", "claude").`

Обсуждение в чат

У вас **gather** на **50 запросов к OpenAI**, Semaphore(10). На 30-м запросе сервер возвращает **429 Too Many Requests**.

Вопрос: какие ваши действия? 3–4 варианта, что делать в production.

Background Tasks: генерация в фоне

Кейс. Пользователь запросил отчёт — генерация 30+ секунд. Не хотим держать HTTP-соединение открытым.

```
example.python

1 import asyncio, uuid
2 from fastapi import FastAPI, BackgroundTasks
3 from openai import AsyncOpenAI
4
5 app = FastAPI()
6 client = AsyncOpenAI()
7 TASKS: dict[str, dict] = {} # в проде — Redis
8
9 async def _generate(task_id: str, topic: str):
10     TASKS[task_id]["status"] = "running"
11     try:
12         r = await client.chat.completions.create(
13             model="gpt-5-mini",
14             messages=[{"role": "user", "content": f"Отчёт: {topic}" }],
15             max_tokens=4000,
16         )
17         TASKS[task_id].update(status="done", result=r.choices[0].message.content)
18     except Exception as e:
19         TASKS[task_id].update(status="failed", error=str(e))
```

Background Tasks: генерация в фоне

```
example.python

1  @app.post("/reports")
2  async def create_report(topic: str, bg: BackgroundTasks):
3      tid = str(uuid.uuid4())
4      TASKS[tid] = {"status": "queued"}
5      bg.add_task(_generate, tid, topic)
6      return {"task_id": tid}  # клиент получает сразу
7
8  @app.get("/reports/{tid}")
9  async def get_report(tid: str):
10     return TASKS.get(tid, {"error": "not found"})
```

Background Tasks: генерация в фоне

Когда это уместно: логирование, webhook. **Не уместно** для критичных задач — рестарт сервиса = потеря очереди.

Типичные ошибки async-кода (1/2)

Три самых частых промаха — все блокируют event loop:

```
example.python

1 # 1. Забыли await — text это coroutine-объект, не строка
2 text = client.chat.completions.create(...) # нет await!
3
4 # 2. time.sleep() вместо asyncio.sleep() — loop замирает
5 async def handler():
6     time.sleep(5) # ВСЕ пользователи ждут 5 секунд
7
8 # 3. requests внутри async — синхронный HTTP блокирует loop
9 async def get_data():
10     return requests.get(url).json() # нужен httpx.AsyncClient
```

Типичные ошибки async-кода (2/2)

Ошибки уровня структуры — код работает, но неправильно:

```
example.python

1 # 4. Semaphore внутри функции — новый счётчик каждый вызов
2 async def analyze_bad(text):
3     sem = asyncio.Semaphore(10) # создаётся на КАЖДЫЙ вызов
4     async with sem: ...        # ничего не ограничивает
5
6 # ПРАВИЛЬНО: Semaphore — атрибут модуля/класса
7 SEM = asyncio.Semaphore(10)
8 async def analyze_ok(text):
9     async with SEM:            # общий счётчик для всех вызовов
10         ...
11
12 # 5. Забыли return_exceptions=True — первая ошибка роняет весь batch
13 results = await asyncio.gather(*tasks) # плохо
14 results = await asyncio.gather(*tasks, return_exceptions=True) # ок
15
16 # 6. Не отменили pending — утечка задач и квоты
17 done, pending = await asyncio.wait(tasks, return_when=asyncio.FIRST_COMPLETED)
18 for t in pending:
19     t.cancel()                 # обязательно!
```

Диаграмма: gather при ошибке

Поведение: падение одной задачи не отменяет остальных. Без `return_exceptions=True` — первая ошибка пробрасывается из `await`, фоновые задачи продолжают тратить квоту.

Когда брать: нужен частичный результат («8 из 10 ок — пойдёт»).



Диаграмма: TaskGroup при ошибке

Поведение: одна задача упала — остальные отменяются автоматически. На выходе из блока — `ExceptionGroup` со всеми ошибками, обрабатывается через `except*`.

Когда брать: нужно всё или ничего (3 параллельных перевода — без всех трёх результат бесполезен).



Вопрос: насколько понятно?

Оцените от 1 до 10: **насколько вы поняли async для LLM?**

Если ниже 6 — напишите в чат, что было непонятно (event loop? Semaphore? TaskGroup? SSE?).

Связь с остальным курсом

Опирается на:

- **M2B3** — ваш синхронный клиент с retry/fallback/кешем. Сегодня переписали его на async.
- **M3B2** — паттерны Streaming, Fan-out/Fan-in, Bulkhead. Сегодня — их код.

Что дальше:

- **B3.4 (FastAPI)**. Async-клиент становится сервисом: `POST /chat`, `POST /chat/stream`, `POST /chat/batch`. DI для роутера провайдеров. Middleware с `asyncio.timeout`.
- **B3.6 (observability)**. Async усложняет трейсинг — нужны correlation-id в contextvars, structlog + Phoenix.
- **M4B4 (Telegram)**. SSE от FastAPI → aiogram через `edit_message_text` по таймеру. «Печатающийся» ответ в Telegram.
- **M5 (RAG)**. Async ускоряет embedding-пайплайн: 1000 чанков → `gather` + `Semaphore` → 10 секунд вместо 10 минут.
- **M6 (агенты)**. LangGraph является async. TaskGroup для параллельных под-агентов. `asyncio.timeout` для ограничения «думающего» цикла.

ИТОГИ

Что вы умеете теперь:

- Объяснять разницу concurrency/parallelism, I/O-bound/CPU-bound
- Писать `async def` + `await`, использовать `AsyncOpenAI` и `AsyncAnthropic`
- Запускать `asyncio.gather` с `return_exceptions=True` и `Semaphore` на N
- Пользоваться `TaskGroup` и `asyncio.timeout` (Python 3.11+)
- Ловить `ExceptionGroup` через `except*`
- Стримить ответ LLM → FastAPI → клиент через `sse-starlette`
- Ставить три уровня таймаутов: HTTP-клиент, логика, middleware
- Бенчмаркать sync vs async и подбирать оптимальный уровень конкурентности

Ключевые цифры:

- 8–10x ускорение на 20 параллельных LLM-запросах
- TTFT 200–600 мс — порог UX для streaming
- Semaphore = $\text{RPM} \times 0.8 / 60$

Домашнее задание (ДЗ-самост)

Переписать синхронный LLM-клиент из М2Б3 на `async`. Войдёт в итоговый проект модуля (чекпоинт 3).

Минимум:

1. Класс `AsyncLLMClient` с методами `complete(prompt)` и `batch_chat(prompts)` через `AsyncOpenAI`.
2. В `batch_chat` — `gather` + `Semaphore(N)`, параметр `N` настраивается.
3. Метод `stream_chat(prompt)` — `async`-генератор, `yield`'ит токены (OpenAI streaming).
4. Эндпоинт FastAPI `POST /chat/stream`.

Критерии сдачи на чекпоинте 3:

- Код работает и использует `AsyncOpenAI` (не `OpenAI`).
- `gather` с `return_exceptions=True`, `Semaphore` — атрибут клиента.
- README с бенчмарком: таблица «sem → время» и обоснование выбранного лимита.

Дополнительные материалы для самостоятельного изучения

Внутренности asyncio (русскоязычные разборы):

- [Habr — «Событийный цикл в asyncio: как Python-код работает поверх механизмов Linux»](#) — разбор реализации цикла событий: ready queue, selector, callbacks.

HTTP-клиенты 2026:

- [Zapros](#) — современный middleware-based HTTP-клиент (cache / retry / mock / VCR).

Референсы

Обязательные:

- [Python asyncio — Coroutines and Tasks](#) — TaskGroup, timeout, gather
- [OpenAI Python SDK — Async Usage](#) — AsyncOpenAI, streaming

Для углубления:

- [OpenAI — Rate Limits Guide](#) — актуальные лимиты по tier'ам
- [MDN — Server-Sent Events](#) — спецификация SSE